Lineage4Agent

A DeepSeek Harness plugin that draws how an agent reached its answer, step by step and subagent by subagent.

Owner **Vinicius Dias** Repo `github.com/Vnidias/Lineage4Agent_DeepSeek` Commit `14e58df` Date **5 November 2026**

PROTOTYPE

Overview

Lineage4Agent (package `dsh-lineage`) is a DeepSeek Harness Web GUI plugin that shows a conversation as a swimlane graph: the prompt, every step and tool call, each subagent and the results that came back. It adds a Lineage tab to every chat and a Lineage button next to Automation tasks. It only reads existing logs.

Contents

1.1 Theme
1.2 Problem to be solved
1.3 Objectives
2.1 General architecture
2.2 Stack
2.3 Data flow and layout
2.4 Module details
2.5 Tests
2.6 Results
2.7 Future improvements
3 Risks and open questions
A Glossary

Problem to be solved

Answers hide their path
The chat shows the final answer, not the chain of steps behind it.

Subagents are scattered
Child agents live in separate chats, so the full tree is hard to follow.

Flat event ledgers
The Trajectory view lists events of one session, with no cross-agent links.

Slow debugging
Finding which step or tool failed means reading long transcripts.

Built with

JavaScript · React 18 (shell) · DeepSeek Harness client plugins · SVG

Objectives

O1 Per-chat lineage
A Lineage tab that draws the open chat as a graph.

O2 Sidebar entry
A Lineage button after Automation tasks to browse any chat.

O3 Subagent branches
Show every subagent at any depth, linked to its parent step.

O4 Step details
Show text, reasoning, tool calls, timing and tokens per card.

O5 Turn focus
Isolate one question and its answer, with the agents that fed it.

O6 Read only
Use existing Host read APIs and never write to a session.

O7 No build toolchain
Ship one bundle with no runtime dependencies.

How it works

1. **Open a chat**
Use the Lineage tab of a chat, or the Lineage button in the sidebar.

2. **Read the logs**
Each agent's log and its subagent list are read.

3. **Fold into steps**
Events become prompt, step, answer and result cards.

4. **Link the agents**
Each subagent is tied to the step that created it.

5. **Inspect**
Click any card for text, tool calls, timing and tokens.

02 Development

Engineering view: how Lineage4Agent is built, run and tested.

2.1 General architecture

The plugin runs only in the browser. Two entry points render the same graph component. A data module reads logs through the shell's typed Remote client, and the Host answers from the session logs on disk.

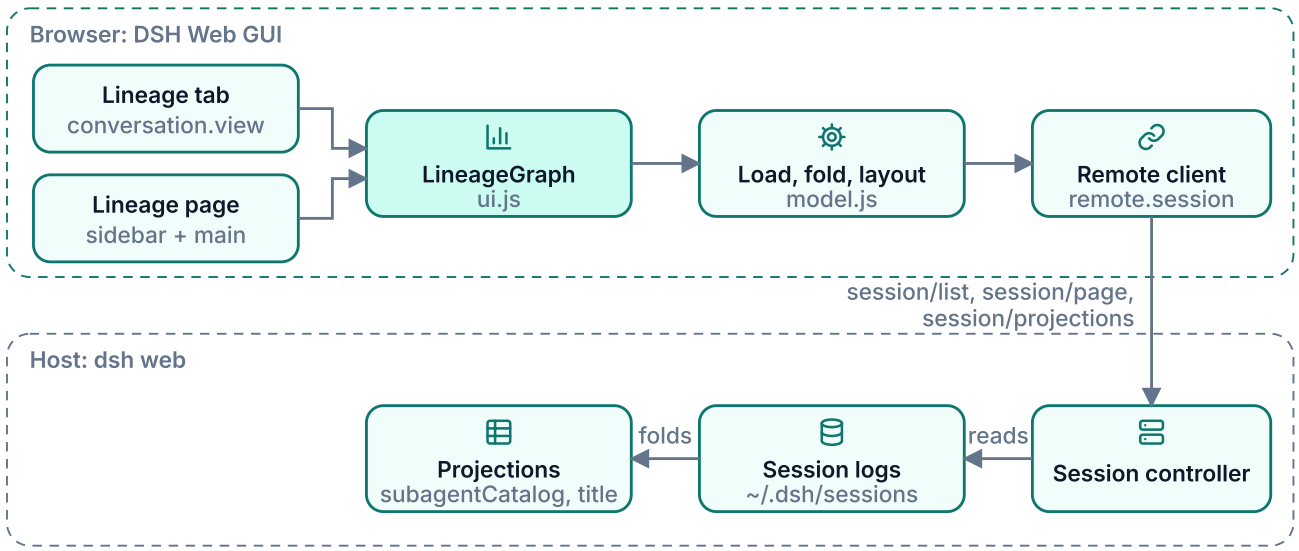


Figure 1. Runtime architecture: two entry points, one graph component and three read-only Host calls.

Layer	Component	Responsibility
Entry points	`conversation.view` tab, `sidebar.panellist` button, `main` page	Where the user opens the view
UI	LineageGraph, Canvas, Detail (ui.js)	Cards, edges, toolbar, detail panel
Data	loadTree, foldSession, buildGraph (model.js)	Read logs, build nodes, lanes, rows and edges
Transport	Remote client of the shell	Typed calls to the Host session controller
Host	Session controller and persistence	Serve log pages and projections from disk

2.2 Stack

JavaScript (ES2022)
4 source modules, no JSX, no compiler

React 18
Taken from the shell's shared module table

DSH client plugin API
Slots, locale, remote.session, uiWorkspace

SVG and CSS
Edges in SVG; styles use --dsw-* tokens

Node.js
Build script and offline test only

jsdom and Chromium
Render test and demo screenshots

Category	Technology	Version
Host	DeepSeek Harness (`@deepseek-ai/dsh`)	0.2.2-rc.1
UI runtime	React (provided by the shell)	18
Plugin package	dsh-lineage	0.1.0
Bundle	lib/client.js	67.8 kB, 19.1 kB gzipped
Test tooling	jsdom, react-dom 18, Node.js	not pinned

The package has no runtime dependencies and no peer dependencies. React is resolved through the shell's module loader, so the bundle does not include it.

2.3 Data flow and layout

> **ADAPTATION** Browser plugin with no jobs or notebooks, so 2.3 describes how logs become a graph and 2.4 describes the source files.

Each time the view opens or refreshes, it runs the same six steps.

1 Resolve the root

In the tab, walk parentSessionId up to the root chat.

2 Load the tree

Read each log with session/page and its children from subagentCatalog.

3 Fold events

Turn 11 event types into prompt, step, answer and result cards.

4 Link agents

Find the step that spawned each child and where its result landed.

5 Lay out

Lanes in depth-first order; rows in time order, below each edge source.

6 Refresh

Reload every 3 s while any agent still has an open turn.

Children of one agent are loaded concurrently, up to 8 levels deep. History seeded into a forked child before its last `session/end-seed` marker is dropped, so a fork shows only its own work. Injected context messages (time, runtime and skill catalogs) and finished steps with no output are hidden.

Order	Rule for the spawning step	Edge style
1	A tool call whose result or arguments contain the child id	Solid blue
2	A delegation call whose run window contains the child's creation time	Solid blue
3	The parent's last card before the child's creation time	Solid blue
Return	A parent message whose sender is the child, else the first parent card after the child ends	Purple dashed (lighter when inferred)

2.4 Module details

The source is 1,575 lines in four modules. The built bundle has 1,599 lines because the build adds a wrapper around them.

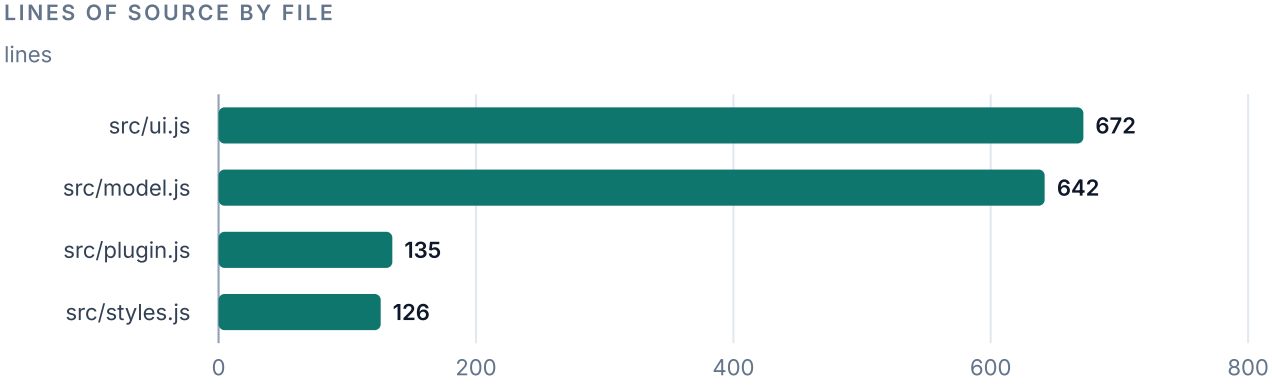


Figure 2. Lines per source file at commit 14e58df.

File	Purpose and key logic	Main exports
src/model.js	Tree loading, event folding, spawn linking, row assignment, totals; no DOM	loadTree, foldSession, buildGraph, totals
src/ui.js	Canvas with pan and zoom, cards, orthogonal edge routing, detail panel, error boundary	LineageGraph, LineageTab, LineagePage, LineageIcon
src/plugin.js	Remote adapter, root resolution, slot registrations, labels in 5 languages	apply, inject
src/styles.js	Theme-aware CSS using the shell's design tokens	STYLES
scripts/build.mjs	Joins the four modules into the DSH ModuleLoader format	(script)
demo/	Standalone page that runs the real bundle on synthetic data	(page)

2.5 Tests

Area	Check	Status
Render test (jsdom)	14 assertions: slots, lanes, edges, detail, links, turn filter, collapse, page	DELIVERED
Offline fold on real logs	test-offline.mjs replays sessions from ~/.dsh/sessions	DELIVERED
Real subagent run	A delegated child in the author's session appears as its own lane with spawn and return edges	DELIVERED
Visual check	Headless Chromium screenshots of the demo, light theme	PARTIAL
Live GUI check	Plugin installed and mounted; not yet viewed in an authenticated browser	NOT MEASURED
Unit tests per function	No test framework	NONE
CI pipeline	No CI configuration	NONE

The render test loads `lib/client.js` the same way the shell does, through a stand-in module loader and a fake client context, so it exercises the shipped bundle rather than the sources. The remaining gap is the live GUI check (see Q1).

2.6 Results

5 / 7
objectives delivered

2 / 7
objectives partial

67.8 kB
bundle, no dependencies

14
render test assertions

ID	Objective	Status	Evidence
01	Per-chat lineage	PARTIAL	Tab registered and render-tested; not yet seen in the live GUI (Q1)
02	Sidebar entry	PARTIAL	Panel button at order 11 after Automation tasks (order 10); same live check pending
03	Subagent branches	DELIVERED	Demo with 5 agents across 3 levels; real child lane from the author's session
04	Step details	DELIVERED	Detail panel shows text, reasoning, tool arguments, results, duration and tokens
05	Turn focus	DELIVERED	Filter keeps the turn plus feeding subagents (11 of 20 cards in the test)
06	Read only	DELIVERED	Only session/list, session/page and session/projections are called
07	No build toolchain	DELIVERED	build.mjs concatenates modules; no runtime dependencies

2.7 Future improvements

ID	Item	Priority	Effort	Why
B1	Confirm both entry points in a live DSH browser session	HIGH	S	O1 and O2 are only render-tested (Q1)
B2	Incremental reads with session/follow instead of full reloads	HIGH	M	Each refresh reads every log in full (R1)
B3	Unit tests for foldSession and buildGraph, plus CI	MEDIUM	M	Only scripted end-to-end checks exist (R4)
B4	Show workflow and code-mode sub-calls as their own cards	MEDIUM	M	Nested calls are summarised in one step
B5	Translate all UI text, not only the tab and button labels	LOW	S	Panel and toolbar text is English
B6	Export the graph as SVG or PNG	LOW	S	Useful for reports and reviews
B7	Publish to npm and the DSH plugin index	MEDIUM	S	Install currently needs a git clone (Q3)

03 Risks and open questions

Known risks and open questions, referenced by stable IDs.

Risks

ID	Risk	Impact	Mitigation	Priority
R1	Very long or deep trees are read in full on every refresh	Slow loading and Host load in live mode	Live refresh runs only while an agent is running; add incremental reads (B2)	MEDIUM
R2	The plugin relies on internal DSH slot and event names	A DSH release can break the view	Error boundary keeps the GUI usable; track DSH releases (Q2)	HIGH
R3	A spawn edge is inferred from timing when no tool result names the child	Edge may point at a nearby step	Rule order prefers explicit ids; inferred returns are drawn lighter	LOW
R4	No unit tests or CI	Regressions found late	Run test-render.cjs before each release; add B3	MEDIUM
R5	The detail panel shows full prompts and tool output	Sensitive text visible on screen	Same data as the chat itself; read only, nothing leaves the Host	LOW

Open questions

ID	Question	Owner	Where it matters
Q1	Do the tab and sidebar button render as expected in a live authenticated DSH page?	Vinicius Dias	2.5 01 02
Q2	Will the conversation.view and sidebar.panellist slots stay stable across DSH 0.2 releases?	Not stated	R2
Q3	Should the plugin be published on npm or a DSH plugin index?	Vinicius Dias	B7
Q4	What tree size should live mode support before incremental reads are required?	Not stated	R1 B2

A Appendix

Reference material.

Glossary

Term	Meaning
DSH	DeepSeek Harness, the agent runtime and Web GUI this plugin extends
Subagent	A child agent started by another agent with subagent, subagent_fork or workflow
Lane	One column of the graph, holding the cards of a single agent
Turn	One prompt or trigger and all the steps until the agent stops
Step	One model call plus the tool calls it made
Projection	A value the Host folds from a session log, such as subagentCatalog
Slot	A named place in the GUI where a plugin can register a component
Bundle patch	The cordis.patch.yml layer that mounts the plugin when it is installed